EGO Planner Refactor

我在构建一个定位+规划耦合的项目。它的名字是iap, 前端是一个GNSS+LiDAR+IMU的里程计，其中有ARAIM模块，用于实时评估GNSS和LiDAR的Protection level. 同时有一个Predictor模块，是一个通用的库，用来查询未来某个时刻指定位置的预测的GNSS和LiDAR的risk level。 还有一个planner模块，参考了 https://github.com/ZJU-FAST-Lab/ego-planner-swarm/tree/ros2_version 的代码，目标是再其基于障碍物规划的基础上增加安全约束，能够让无人机走最安全的轨迹。这时就需要在planner中调用predictor模块，iap_report_v2.0中是原来的方案，增加了一个risk grid map, 新增了一个Astar计算初解的流程，但是对planner改动太大了，不一定是最好的方案。请你帮我梳理ego-planner的整个流程，哪里适合让我的安全概念其作用？该怎么有机结合安全的概念？

https://github.com/ZJU-FAST-Lab/ego-planner-swarm/tree/ros2_version

Claude-Opus-4.8

思考中

下面我按照"先梳理流程、再定位注入点、最后给出推荐方案和落地细节"的顺序来回答。我会基于 ego-planner（EGO-Planner: ESDF-free gradient-based local planner，以及它的 swarm/ROS2 分支）的标准架构来讲，你的 fork 里函数名可能略有出入，但整体骨架是一致的。

一、ego-planner 的整体流程

ego-planner 可以分成三层：**FSM 调度层 → Planner Manager 重规划层 → B-spline 优化层**。

1. **FSM 调度层（`EGOReplanFSM`）**

这是一个状态机，典型状态有 `INIT / WAIT_TARGET / GEN_NEW_TRAJ / REPLAN_TRAJ / EXEC_TRAJ / EMERGENCY_STOP`。它负责：接收目标点、按固定周期触发重规划、监控当前执行

轨迹是否还安全（碰撞检查），以及在异常时切到紧急停止。真正"决定要不要重新规划、什么时候重新规划"的逻辑在这一层。

2. **Planner Manager 重规划层（`EGOPlannerManager::reboundReplan`）**

这是一次规划的主入口，流程大致是：

- 先生成一条**初始轨迹**（initial guess）。基础版 ego-planner 这里很轻：从全局参考轨迹或一条多项式/直线采样出一组点，参数化成一条均匀 B-spline 的控制点。注意：ego-planner 的"前端"非常薄，它没有 Fast-Planner 那种 kinodynamic A* 搜索作为强前端，初值质量主要靠后端去救。
- 调用后端优化 `BsplineOptimizer::BsplineOptimizeTrajRebound`。
- 优化成功后做可行性检查、必要时 `refine`（重新拟合 + 时间重分配），最后输出 B-spline 轨迹给控制器。

3. **B-spline 优化层（`BsplineOptimizer`）——这是 ego-planner 的灵魂**

ego-planner 的核心创新就在这里：它**不建 ESDF**。当 B-spline 与障碍物相撞时，`initControlPoints` 会对每个碰撞控制点，沿碰撞方向向外搜索，找到一个障碍表面锚点 `p` 和一个排斥方向 `v`，构成一组 `{p, v}` 对（存在 `cps_.base_point` 和 `cps_.direction` 里）。然后用 L-BFGS 最小化一个组合代价（`combineCostRebound`）：

- `calcSmoothnessCost`：基于控制点的 jerk/弹性带平滑代价；
- `calcDistanceCostRebound`：用 `{p,v}` 对算出来的避障代价（控制点离锚平面太近就受罚）；
- `calcFeasibilityCost`：速度/加速度限幅。
- （swarm 版还有 `calcSwarmCost`。）

代价合并为 `f = λ1·smooth + λ2·dist + λ3·feasi`，对**内部控制点**求梯度做 L-BFGS。所有代价都在**控制点**上评估（利用 B-spline 凸包性质：控制点安全则曲线安全）。

这个架构的关键含义是：**ego-planner 的安全完全是"在一个固定的同伦类里、靠梯度把曲线推出障碍"**。它不做全局拓扑搜索，不会主动换一条走廊。

二、安全概念可以注入的位置（逐个评估）

把你的 predictor（给出某位置某时刻的 advisory PL / IM / 风险代价 `c_PI`，且可查询梯度）往里塞，理论上有五个口子：

(A) 后端 B-spline 优化的代价项里——加一个 `calcIntegrityCost`，和 `calcDistanceCostRebound` 完全并列。这是**最贴合 ego-planner 哲学的注入点**，因为 ego-planner 整个设计就是"往控制点上叠加代价项"。改动最小、最有机。

(B) 前端初值生成里——这就是你 v2.0 加 A* 的地方。给初值一个 risk-aware 的拓扑/走廊选择。

(C) FSM 的重规划触发 / 可行性门里——当预测的 integrity margin 变负或低于阈值时，触发重规划、降速、悬停或返航（对应你 report 里的 fallback 逻辑）。

(D) 时间分配 / 速度——在低 integrity 区域降速（预测窗口 `t+τ` 内放慢，让定位有时间收敛）。

(E) 目标 / 全局参考路径选择——让全局 waypoint 走高 integrity 走廊。

我的核心判断是：**(A) 应该是主力，(C) 必须有（很便宜），(B) 是可选的、且不该做成你 v2.0 那样的强制 A* 新阶段。** 下面解释为什么。

三、推荐方案：以"后端软代价"为主，前端轻量化

你担心 v2.0"对 planner 改动太大"，这个直觉是对的。问题不在于"加了 risk grid map"——那个其实是需要的（见下文梯度问题）——*问题在于把 A 做成了一个强制的新前端阶段。这破坏了 ego-planner"薄前端 + 强后端"的结构，引入了同伦类切换、A 与* B-spline 初值不一致、调参复杂度等一堆新麻烦。

更有机的做法是分两阶段：

阶段一（先做、改动最小）：后端 integrity 软代价

在 `combineCostRebound` 里加一项：

f=λ1Jsmooth+λ2Jdist+λ3Jfeasi+λPIJintegrityf = \lambda_1 J_{\text{smooth}} + \lambda_2 J_{\text{dist}} + \lambda_3 J_{\text{feasi}} + \lambda_{\text{PI}} J_{\text{integrity}}f=λ1Jsmooth+λ2Jdist+λ3Jfeasi+λPIJintegrity

其中对内部控制点 QiQ_iQi：

Jintegrity=Σ icPI(Qi, ti),∂ Jintegrity∂ Qi=∇ p cPI(Qi, ti)J_{\text{integrity}} = \sum_i c_{\text{PI}}(Q_i,\, t_i), \qquad \frac{\partial J_{\text{integrity}}}{\partial Q_i} = \nabla_p\, c_{\text{PI}}(Q_i,\, t_i)Jintegrity=i Σ cPI(Qi,ti),∂Qi∂ Jintegrity=∇pcPI(Qi,ti)

其中 tit_iti 是该控制点在轨迹上对应的时间，传给 predictor 作为 fpred(p, t+τ)f_{\text{pred}}(p,\, t+\tau)fpred(p,t+τ) 的查询时间——**这恰好让你"预测未来位置风险"的设计真正用上了**，而不是退化成 quasi-static 查询。

这样做的好处：

- 改动局限在 `BsplineOptimizer` 一个函数 + 一个新 cost 函数，FSM 和 Manager 几乎不动。
- 完美复用 ego-planner 的 L-BFGS 框架和控制点梯度索引（注意保持和现有代码一样的 `order_` 边界偏移约定，只对被优化的内部控制点写梯度）。
- 行为上得到"在当前走廊内，贴着高 integrity 的一侧走、并在风险区降速"的效果，足以作为论文里"planner 确实用上了 PL"的第一个可展示结果。

阶段二（按需）：轻量 risk-aware 前端

只有当出现这种情况时才需要前端：**避开低 integrity 区需要换一个同伦类/走廊**（比如要绕到障碍物另一侧、或选另一条街），而纯梯度后端会卡在风险"山谷"里出不来。这时再加前端，但要轻量化：

- 不要让 A* 替换 ego-planner 初值，而是让 A*（在一个**粗**的 occupancy + risk 联合代价栅格上）**只用来挑同伦类/走廊**，输出一条引导路径作为 ego-planner 的 initial guess，后端仍然主导细化。
- 或者更省事：让你的**全局参考路径**（喂给 ego-planner 的 reference）走高 integrity 走廊，前端逻辑放到全局层，局部 ego-planner 完全不动。

也就是说：A* 可以保留为"可选的初值偏置器"，而不是"强制新阶段"。

四、几个必须想清楚的设计点

4. 梯度从哪来 → 这才是 risk grid 真正的用途

ego-planner 的代价梯度都是解析的，而 predictor 只给标量 `c_PI`。你需要 ∇pcPI\nabla_p c_{\text{PI}}∇pcPI。每个控制点做有限差分要查 predictor 7 次（中心差分 3 维），一次优化几十个控制点 × 几十次 L-BFGS 迭代，predictor 单次 ~30μs，会被放大到不可接受。所以**缓存一个局部 risk 场（你 report 里的 URG/PL map），用三线性插值取值 + 取梯度**，是合理且必要的。

关键结论：*risk grid 该留，但它的定位应该是"后端梯度查询表"，而不是"A 的地图"**。这样它就从一个重型新模块降级成一个轻量缓存层，正好和你 Predictor Refactor Proposal 里"Predictor 只查询、Safety Grid 负责缓存/插值、Planner Adapter 负责 cost"的分层一致。

5. PL 还是 IM？要不要把 AL = f(d_obs) 拉进来？

这里有个对 ego-planner 特别重要的简化机会。你 report 里 `IM = AL − PL`，而 `AL = min(γ_H d_obs, γ_V d_vert)` 需要障碍物距离场——但 **ego-planner 故意不维护 ESDF**。如果为了算 AL 又把 ESDF 建回来，等于否定了 ego-planner 的核心优势。

更干净的解耦是：**障碍物间距已经由 ego-planner 的 `calcDistanceCostRebound` 负责了**，你没必要再把 `d_obs` 通过 AL 塞进 integrity 项里。让 integrity 项**只惩罚定位侧的 `PL_pred`**（偏好低 PL 区域），把"离障碍多远"留给原有的碰撞项。两个 cost 各管一件事、互不重复，耦合度最低，也最符合"定位安全 vs 障碍安全是两个独立关切"的物理直觉。AL/IM 这套统一语言更适合放在 **FSM 的监督/可行性判定**里（见下一点），而不是放进梯度代价里。

6. FSM 层的硬门 + fallback（很便宜，强烈建议做）

在 `EGOReplanFSM` 的轨迹可行性检查里，除了现有碰撞检查，加一个 integrity 门：沿规划出的轨迹采样，若 mintIM(p(t),t)<0\min_t IM(p(t), t) < 0mintIM(p(t),t)<0（即 PLpred>ALPL_{\text{pred}} > ALPLpred>AL）或 predictor 报 `available=false`，则：触发重规划 / 降速 / 悬停。这正好对接你 report 第 3.5.8 节的 stale/unknown-risk fallback，也对接 predictor 实验六的"GNSS-required outage"行为——**当 predictor 返回 NaN/unavailable，planner 不能把它当 0 风险，必须走保守分支**。这一块改动极小但对"系统级安全闭环"叙事很关键。

7. 局部极小与梯度冲突

risk 场是个平滑场（类 ESDF），叠到 ego-planner 上会引入新的局部极小，且 integrity 梯度可能和碰撞梯度方向打架（比如贴墙处 LiDAR 反而观测好，但碰撞项要把你推离墙）。务必：

- 给 `λ_PI` 做软启动/退火，或先收敛碰撞+平滑、再加 integrity；

- 对 `c_PI` 做归一化和饱和（hinge 形式，只在 IM 接近 0 时才强惩罚，像你 report 的 JPIJ_{PI}JPI 那样），避免在本来就安全的区域产生无谓梯度去对抗碰撞项；
- 保留 ego-planner 原有的 rebound/碰撞优先级，让 integrity 是"次序更低的偏好项"。

8. **predictor 调用频率与缓存**

后端一次优化会查很多控制点。建议 risk 场按 ego-planner 重规划频率（5–10 Hz）异步刷新一次局部缓存，优化迭代内只查缓存、不实时调 predictor；这和你实验十二（batch latency stress，100-query/tick p95 ~16ms）的结论一致——批量查询可行，但放进 L-BFGS 内层循环仍然太贵。

五、小结：建议的注入图

代码块

```
全局参考路径（可选 risk 偏置）          ← 阶段二/E（可选）
        │
EGOReplanFSM
  ├─ 重规划触发 + integrity 可行性门 + fallback    ← (C) 必做，便宜
  │
EGOPlannerManager::reboundReplan
  ├─ 初值生成（可选 risk-aware 引导，非强制A*）    ← (B) 按需，轻量
  │
BsplineOptimizer::combineCostRebound
  ├─ smoothness
  ├─ distance(障碍，原有)          ← 障碍安全归它
  ├─ feasibility
  └─ + integrity(PL_pred, t_i)    ← (A) 主力，最有机
        ↑ 用局部缓存 risk 场(URG)取值+梯度   ← grid 的真正定位
```

一句话：*把安全主要做成后端的一个软代价项（A），用一个轻量缓存场提供梯度，再在 FSM 加一道 integrity 硬门和 fallback（C）；A 前端（B）降级为可选的初值偏置而不是强制新阶段。* 这样既保住了 ego-planner"薄前端 + 强后端 + 无 ESDF"的结构优势，又让你的 PL/integrity 概念真正进入规划，比 v2.0 的大改方案更克制也更容易调通和写进论文。

如果你愿意贴一下你 fork 里 `bspline_optimizer.cpp` 的 `combineCostRebound` 和控制点梯度索引部分，我可以帮你把 `calcIntegrityCost` 的具体接口、梯度写入下标和 `λ_PI` 退火逻辑落到能编译的层面。

16:08

我觉得你说的很好，关于risk grid map怎么设计我还不清楚。是该把它嵌套进ego现有的grid map中还是再维护一个新的？关于初值的计算方式我还有疑问，可以详细解释下这两个方式的流程吗？需要对ego进行哪些改动？1） 不要让 A* 替换 ego-planner 初值，而是让 A*（在一个粗的 occupancy + risk 联合代价栅格上）只用来挑同伦类/走廊，输出一条引导路径作为 ego-planner 的 initial guess，后端仍然主导细化。2） 或者更省事：让你的全局参考路径（喂给 ego-planner 的 reference）走高 integrity 走廊，前端逻辑放到全局层，局部 ego-planner 完全不动。

参数 · 16:26

Claude-Opus-4.8

思考中

Good questions — these two are exactly where the "clean vs. messy" line gets drawn. Let me take them one at a time.

Part 1: How to design the risk grid map

First, a fact about ego-planner that drives this decision: ego's `GridMap` (in `plan_env`) is an **occupancy** map. It maintains a probabilistic occupancy buffer plus an inflated occupancy buffer, updated by raycasting from depth/pointcloud at high rate (roughly 5–20 Hz). It deliberately does *not* maintain an ESDF. The optimizer only ever asks it `getInflateOccupancy(pos)`. So ego's map is semantically "is this voxel blocked," updated fast, at fine resolution (~0.1–0.15 m).

Your risk field is a completely different animal: it's a localization-side quantity (`PL_pred`), it changes slowly in space, it's expensive to compute, and you want to refresh it at 1–2 Hz over *active voxels only*. Trying to cram that into ego's occupancy buffer means two things with different resolutions, different update rates, and different lifecycles fighting over one data structure.

My recommendation: maintain a separate risk grid class, but align its frame and indexing conventions with ego's `GridMap`. Concretely:

- **Separate buffer, same world frame.** This matches your Predictor Refactor Proposal exactly: Predictor only queries, `SafetyGridBuilder` caches/interpolates, `PlannerAdapter` turns it into cost. Nesting it into ego's `GridMap` would re-couple the very layers you just argued should be separated.

- **Coarser resolution than occupancy.** Risk varies slowly, so 0.5–1.0 m voxels are fine, and coarser means far fewer predictor queries per refresh. The two maps do *not* need matching resolution — they're queried independently by world position. They only need the same world frame (and ideally the same rolling-window center update, which you can share).
- **Rolling local window**, centered on the UAV, same idea as ego's local map update region. This is your report's Gt\mathcal{G}_tGt.
- **Per voxel, store**: `c_PI` (precomputed planner cost, or `PL_pred` + compute `c_PI` on read), an age/timestamp, and a validity flag. Storing `c_PI` directly is what lets you skip the predictor in the inner loop.

The reason this matters for the backend cost (阶段一 from before): the optimizer's L-BFGS needs `c_PI(p)` **and** `∇_p c_PI(p)` at many control points per iteration. If you finite-difference the predictor, that's ~7 predictor calls per control point per iteration — fatal. With a cached grid you do **trilinear interpolation** for the value, and the **analytic gradient of the trilinear interpolation** for `∇_p c_PI` — both are a handful of array lookups. So the grid's real job is "be the optimizer's gradient lookup table," not "be A*'s map." That demotes it from a heavy new subsystem to a thin caching layer.

So the answer to "nest or separate": **separate class, shared frame, coarser resolution, exposing `getRiskCost(pos)` and `getRiskGrad(pos)`.** Give the optimizer a tiny adapter that can ask occupancy from `GridMap` and risk from `SafetyGrid` by position.

Part 2: How ego actually builds its initial guess (so the two methods make sense)

You can't see where to inject without knowing ego's current flow, so here it is.

There are two "front-end-ish" stages in stock ego-planner, and they're both very thin:

Stage A — the global reference, generated once per target. When the FSM receives a new goal, it calls something like `planGlobalTraj` / `planFromGlobalTraj`. This fits a coarse smooth trajectory (typically a min-snap polynomial through a few waypoints, sometimes just start→goal). This global traj is *not* collision-aware in any deep sense — it's a reference.

Stage B — the local initial control points, rebuilt every replan. Inside `EGOPlannerManager::reboundReplan` (or the FSM helper that feeds it), ego samples points along the global reference between the current start and a *local target*, at a fixed time step tst_sts, producing a point sequence `point_set`. Then `UniformBspline::parameterizeToBspline(t_s, point_set, start_end_derivatives, ctrl_pts)` converts that sequence into uniform B-spline control points. Those control points are the initial guess handed to the optimizer.

That's it. The "front end" is literally: *fit a smooth global curve once, then resample it locally into control points.* The heavy lifting — pushing the curve out of obstacles — is all in the backend

optimizer. This is why I keep saying ego is "thin front-end, strong back-end."

Now the two methods map cleanly onto these two stages.

Method 1: A* picks the corridor, feeds the local initial guess (touches Stage B)

Flow, per replan:

1. You have `start_pt` and the `local_target`.
2. Build a coarse **joint cost** over local cells: `cost(cell) = ∞` if occupied/inflated (from ego's `GridMap`), else `cost(cell) = base_step + λ · risk(cell)` (risk from your `SafetyGrid`).
3. Run A* (or Dijkstra) `start → local_target` on that joint grid → a grid path.
4. Resample that path uniformly in time/length → this becomes `point_set`.
5. `parameterizeToBspline(point_set)` → initial control points.
6. Optimizer runs exactly as before (ideally also with the backend integrity cost).

Changes to ego: add `getRiskAwareInitPath(start, target)` in the manager; add a small local A*; in `reboundReplan`, use this `point_set` when A* succeeds and the grid is fresh, otherwise **fall back to the stock global-reference sampling**. The optimizer and FSM are essentially untouched.

What it buys you: the initial guess can start in a *different homotopy class / corridor* than the global reference — i.e., it can route around to the low-PL side of an obstacle, which pure gradient descent in the backend can never do (the backend can't cross an obstacle to find a better basin).

The cost / danger: this is exactly the complexity I warned about in your v2.0. Because A* can pick a different corridor each replan, consecutive replans can jump between homotopy classes, causing the trajectory to flip back and forth. You then need consistency handling — seed A* with the previous path, add hysteresis, or only switch corridors when the integrity gain exceeds a threshold. This is real work and real tuning. So Method 1 is the right tool **only when you genuinely need to change corridors for integrity**, not as a default.

Method 2: the global reference walks a high-integrity corridor (touches Stage A only)

Flow, once per new target (not every replan):

1. Build a coarse global graph/grid over the known map, with risk baked in.
2. Search `start → goal` through low-`PL` regions → a waypoint sequence.
3. Fit the usual smooth global trajectory (min-snap) through those waypoints.

4. The local ego planner replans against this reference *with no changes at all*.

Changes to ego: only the waypoint selection inside `planGlobalTraj`. Local planner, optimizer, FSM control flow — all untouched.

What it buys you: the whole mission biases toward high-integrity corridors (your report's "urban canyon: take the longer, open-sky route B"), and you got it almost for free.

The cost / danger: the global search needs a *global-ish* risk map, which may be partly unknown ahead of the drone. And because the global traj is only regenerated on a new target (or on large deviation), it's not reactive to *newly discovered local* risk. But — and this is the key point — **the backend integrity cost (阶段一) handles exactly that local reactivity.** So Method 2 and the backend soft cost are complementary and clean.

Putting it together — my concrete recommendation

Do this ordering:

1. **Backend integrity soft cost** (the `calcIntegrityCost` from the previous answer), fed by the **separate cached risk grid** with trilinear value+gradient. This alone gives "hug the high-integrity side within the current corridor, and slow down in risky regions," and it's the smallest, most ego-native change.
2. **Method 2** for global biasing — it's nearly free and gives the "choose the safe corridor at mission scale" behavior. Local ego stays literally unchanged.
3. **Method 1 only if** you empirically see the backend getting trapped in a risk valley that requires a *local* corridor change. Then add A* as a **conditional initial-guess biaser with a fallback to stock sampling** — never as a mandatory new stage. And budget time for homotopy-consistency tuning.

The throughline: *the risk grid is a gradient cache (separate class, shared frame, coarse), not a planning map; Stage A and Stage B are your two cheap injection points; and you reach for A last, scoped narrowly, not as a front-end replacement.** That keeps ego's "thin front-end, strong back-end, no ESDF" structure intact while genuinely putting `PL` into the planner.

If you paste your fork's `planGlobalTraj` (for Method 2) and the `point_set` / `parameterizeToBspline` block in `reboundReplan` (for Method 1), I can show you the exact lines to change and where the fallback branch should sit.